Distribution-Enhanced Knowledge Distillation

Calvin Kinateder[1*], Usman Anjum[2*] and Justin Zhan[1]

[1*]Dept. of Computer Science, University of Cincinnati, 2600 Clifton Ave, Cincinnati, 45221, OH, USA.
[2]Dept. of Computer Science, Ottowa University, 15950 N Civic Center Dr, Surprise, 885374, AZ, USA.

*Corresponding author(s). E-mail(s): kinatecj@mail.uc.edu; usman.anjum@ottawa.edu;
Contributing authors: zhanjt@ucmail.uc.edu;

Abstract

The Tsetlin Machine (TM) is a framework that uses propositional logic to learn patterns from data by creating human-interpretable conjunctive clauses. Similar to neural networks, TM accuracy generally increases with model size, and larger clause sets lead to slower training and higher memory usage. Knowledge distillation (KD) is widely used in neural networks to transfer information from a large teacher model to a smaller student model, improving accuracy without increasing training time. Extending KD to Tsetlin Machines is not straightforward due to the absence of differentiable logits. Consequently, we propose Distribution-Enhanced Knowledge Distillation (DKD), a TM-based framework that adapts KD principles to TMs. DKD transfers knowledge through two mechanisms: (1) an intelligent clause-selection algorithm that identifies and initializes the student with the most informative teacher clauses, and (2) a probability-based distillation scheme that generates soft distributions from the teacher's unclamped class sums and incorporates them into the student's training feedback. Together, these components allow the student to learn higher-level decision patterns normally accessible only to a larger TM. Experiments on benchmark image and text datasets demonstrate that DKD consistently improves student accuracy while maintaining inference time when compared to a parametrically identical baseline. This approach effectively narrows the performance gap between teacher and student models, offering a practical path toward deploying compact and interpretable Tsetlin Machines in resource-constrained environments.

Keywords: Tsetlin Machine, Knowledge Distillation, Explainable AI, Propositional Logic, Classification, Data Mining

1 Introduction

Deep neural networks have become central to solving problems in regression, object recognition, text generation, and many other domains. As computational capability has advanced, these networks have grown deeper and more complex. Increased complexity, however, reduces interpretability, raises training cost, and slows inference speed, particularly on embedded or resource-constrained hardware. Knowledge distillation, introduced by Hinton et al. in 2015, seeks to address these challenges by transferring the behavior of a large teacher model to a smaller student model while maintaining comparable performance [1].

Knowledge distillation operates through a teacher-student paradigm in which the student model learns to imitate the teacher's internal representations and decision patterns. Neural networks lend themselves naturally to this approach because their hidden layers encode structured representations that serve as a form of "knowledge." During distillation, the student is trained to minimize the difference between its outputs and the softened output distributions of the teacher, a process shown empirically to improve generalization [2]. Temperature scaling plays a central role in this process by smoothing the teacher's logits before the softmax transformation, thereby exposing inter-class similarity information that aids student learning [1, 3].

The Tsetlin Machine (TM), introduced by Granmo in 2018 [4], represents a different class of learning algorithms built on Tsetlin Automata (TA) and propositional logic. TMs learn collections of human-interpretable clauses and have demonstrated strong performance on classification and regression tasks [5, 6]. Their rule-based structure provides transparency and aligns naturally with goals in explainable AI [7, 8]. Additionally, logic-based algorithms can offer faster execution on hardware and similar propositional logic-based approaches have been effectively used in other machine learning contexts [9, 10].

Using propositional logic in Tsetlin Machines has many unique advantages over traditional machine learning algorithms. Propositional logic based learning is more interpretable and explainable, making it easier for humans to understand. Because of this, Tsetlin Machines have numerous important applications in explainable AI (xAI), a field where transparency and interpretability are of chief importance in AI algorithms [7, 8]. Tsetlin Machines can also achieve a much faster execution time over traditional machine learning algorithms that require advanced vector multiplication. Propositional logic based learning algorithms have been utilized by traditional machine learning algorithms [9, 10].

Although TMs have shown substantial promise, model accuracy generally increases with the number of clauses, making large TMs computationally expensive and difficult to deploy on edge devices or FPGAs [11]. These limitations naturally raise the question of whether knowledge distillation techniques can be adapted to the discrete, non-differentiable structure of Tsetlin Machines. Traditional distillation methods cannot be applied directly, as TMs do not produce logits, feature maps, or differentiable intermediate representations.

In this work, we explore knowledge distillation for Tsetlin Machines by introducing a framework that transfers information through clause-level structure and distributional signals derived from class sums. We make the following contributions:

- We introduce Distribution Enhanced Knowledge Distillation (DKD) for Tsetlin Machines (TM), a distillation framework that transfers the full class confidence distribution derived from unclamped class sums rather than only hard class predictions.
- We introduce a numerically stable transformation of discrete and potentially unbounded TM class sums into probability distributions suitable for distillation. Unlike weighted TMs, which modify clause weights, or regularized TMs, which alter training penalties, DKD operates entirely at the supervision level.
- We combine distribution level supervision with structured clause initialization, enabling the student to inherit both structural bias and distributional confidence information from the teacher.
- We provide empirical evaluation across image and text datasets demonstrating that distribution based supervision improves student performance compared to training without distillation.

2 Related Works

Knowledge distillation (KD) was introduced by Hinton et al. in 2015 as a strategy for improving model generalization while reducing model size and computational cost [1]. Gao et al. later proposed a two-stage distillation framework that transfers backbone representations from teacher to student before task-specific training, substantially narrowing the accuracy gap between the two models [12]. KD has since been applied across many domains. In natural language processing (NLP), distillation using large unlabeled transfer sets drawn from pre-trained language models has enabled compact student models to generalize effectively and even outperform larger architectures such as OpenAI's GPT when combined with bidirectional LSTMs [13–15]. Other variants include self-distillation, where a single model improves its own representations through iterative shrinking [16], and approaches that initialize student models from teacher weights to accelerate training and improve performance [17].

The Tsetlin Machine (TM), introduced by Granmo in 2018 [4, 18, 19], offers a rule-based learning framework grounded in Tsetlin Automata. Numerous extensions have expanded the capabilities of the TM. Contextual bandit formulations leverage algorithms such as Thompson sampling and ϵ-greedy strategies [20]. Convolutional TMs adapt clause concepts from classical convolutional neural networks for image classification [21–23]. Regression-based TMs extend the architecture to produce continuous outputs using specialized voting mechanisms [24].

Other architectural enhancements include the coalesced TM, which supports multi-output learning and jointly optimizes clause composition and weighting through stochastic search [25, 26]. TMs have also shown promise in natural language processing for text encoding, document classification, and sentiment analysis [27–31]. Additional improvements include the label-critical TM, which uses paired Tsetlin Automata to refine label assignment [32].

Several works focus on improving efficiency and generalization. Weighted TMs assign clause-specific weights to emphasize discriminative patterns and reduce computational overhead [33]. Regularized TMs introduce moving-average and weighted-average regularizers to stabilize training and mitigate overfitting, and further explore softmax-based classification as an alternative to raw clause sums [34, 35]. Multi-layer TM architectures demonstrate that hierarchical clause learning can improve performance on complex datasets [36]. Other variants involves the inclusion of an Absorbing Automata [37] and incorporating ways to limit the clauses [38, 39].

Existing Tsetlin Machine extensions such as weighted TMs and regularized TMs modify internal clause weighting mechanisms or introduce regularization strategies to improve generalization. In contrast, the proposed DKD framework does not alter the internal learning rules or clause update dynamics of the Tsetlin Machine. Instead, it introduces a teacher–student supervision mechanism that transfers distributional information derived from class sums. While conceptually inspired by knowledge distillation in neural networks, the algorithmic realization is specific to the discrete clause voting structure of Tsetlin Machines and requires a dedicated transformation of class sums into stable probability distributions.

3 Preliminary: Tsetlin Machines

In a Tsetlin Machine, propositional logic is used to form clauses composed of literals derived from the input features. For binary input data, each feature produces two literals, the feature itself and its negation, giving a total of $2n$ literals for an n-dimensional input vector. Each literal is controlled by a Tsetlin Automaton (TA), which determines whether that literal is included in or excluded from a clause during training.

A clause is expressed as a conjunction of the literals selected by its automata. During inference, the clauses collectively vote for a class: clauses with positive polarity support the target class, whereas negatively polarized clauses vote against it. The final prediction is obtained by comparing the aggregated positive and negative votes through a thresholded decision rule. Figure 1 shows a high-level overview of the inference process for a binary-output Tsetlin Machine.

Let the input vector be

$$X = (x_1, x_2, \ldots, x_n) \in \{0, 1\}^n.$$

The full set of literals is

$$L = \{x_1, \ldots, x_n, \neg x_1, \ldots, \neg x_n\},$$

so that the number of literals is $|L| = 2n$. A clause C_j is defined as a conjunction over a subset of literals,

$$C_j = \bigwedge_{l_k \in L_j} l_k,$$

where $L_j \subseteq L$. Each clause evaluates to $C_j \in \{0, 1\}$ depending on whether all included literals are satisfied by the input.

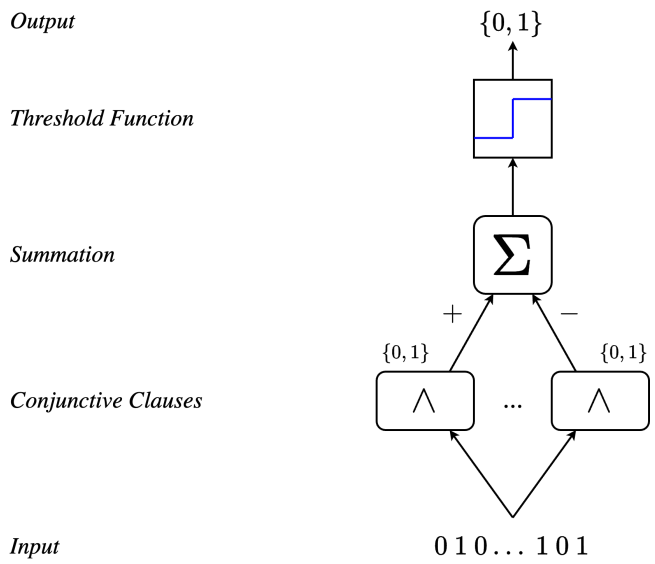


Fig. 1: Inference structure for a binary-output Tsetlin Machine.

Clauses are evenly divided into positive and negative polarity groups. Positive clauses vote for the target class, whereas negative clauses vote for the opposite class. For the weighted multi-class Tsetlin Machine (WMCTM) used in this work, the predicted class among ζ possible labels is computed as

$$\hat{y} = \underset{i=1,\ldots,\zeta}{\operatorname{argmax}} \left(\sum_{j=1}^{|C|/2} w_j^+ C_j^+ - \sum_{j=1}^{|C|/2} w_j^- C_j^- \right),$$

where w_j^+ and w_j^- denote the learned clause weights for positive and negative polarity clauses, respectively.

Each literal is governed by a Tsetlin Automaton with $2N$ internal states. States $1, \ldots, N$ correspond to excluding the literal from the clause, and states $N+1, \ldots, 2N$ correspond to including it. The automaton transitions between states through Type I and Type II feedback.

Type I feedback reinforces patterns associated with correct classifications. It encourages clauses to include literals that frequently co-occur with the true class, while discouraging literals that introduce noise. The specificity parameter s determines the balance between include-focused (Type IA) and exclude-focused (Type IB) updates.

Type II feedback is activated when a clause contributes to a false positive. In such cases, the automata of included literals transition toward states that reduce the clause's likelihood of activating on similar incorrect inputs, thereby improving class discrimination.

Through these local, automaton-based updates, Tsetlin Machines learn structured logical patterns without relying on gradients or backpropagation. Throughout this

work, we employ weighted multi-class Tsetlin Machines, as clause weighting has been shown to improve both discrimination and training behavior.

4 Methodology

In this section, we describe how knowledge distillation can be adapted to the Tsetlin Machine framework. We propose *Distribution-Enhanced Knowledge Distillation* (DKD), a method that transfers information from a larger teacher Tsetlin Machine to a smaller student by leveraging both clause-level structure and distributional class information. Although Tsetlin Machines differ fundamentally from neural networks, DKD is inspired by classical distillation methods by using softened output distributions to guide learning.

The central idea behind DKD is that the teacher's most informative patterns should be transferred in a manner that improves generalization without increasing model size. Our approach integrates two complementary mechanisms: (1) *clause initialization*, in which the student is seeded with a subset of influential clauses from the teacher, and (2) *distributional supervision*, where the student receives additional guidance through probability distributions derived from the teacher's class-sum outputs.

To control the relative influence of these components, DKD introduces three interpretable hyperparameters:

1. **Temperature** ($\tau \in (0, \infty)$): Regulates the entropy of teacher-derived probability distributions. High temperatures ($\tau > 1$) produce softer distributions that highlight inter-class similarities, while low temperatures ($\tau < 1$) sharpen the distribution around the predicted class. It is adapted from traditional knowledge distribution [1].
2. **Balance** ($\alpha \in [0, 1]$): Determines the trade-off between ground-truth labels and teacher soft labels. When $\alpha = 1$, training reduces to standard TM learning; when $\alpha = 0$, training relies entirely on soft labels.
3. **Weight transfer fraction** ($z \in [0, 1]$): Controls how clauses are selected during initialization. Larger values emphasize high-weight clauses, and smaller values promote structural diversity among transferred clauses.

We denote the teacher Tsetlin Machine as TM_T, the baseline (trained without distillation) as TM_B, and the student as TM_S. The models TM_B and TM_S are parametrically identical, enabling direct comparison of performance with and without distillation.

Traditional knowledge distillation in neural networks relies on logits or hidden-layer activations obtained through backpropagation. Tsetlin Machines do not produce such continuous internal representations. To address this limitation, we construct probability distributions from the teacher's unclamped class sums and combine them with clause transfer, resulting in a hybrid feature- and response-based form of distillation that enhances student accuracy while preserving interpretability and low latency [17, 34].

Following the standard KD workflow, the teacher is first trained for E_T epochs. The student is then initialized with clauses selected from the teacher and trained for E_S epochs using both hard labels and teacher-derived soft labels.

A key advantage of this approach is that the student model does not require the teacher during inference. The final student model is therefore suitable for deployment in memory- and power-constrained environments, including embedded systems and FPGA implementations where the larger teacher cannot be feasibly stored or executed.

4.1 Clause Initialization

The first component of our hybrid distillation framework operates prior to training the student model. Let the trained teacher Tsetlin Machine have a clause set C_T and the student have a clause set C_S, denoted as $TM_T \ni C_T$ and $TM_S \ni C_S$, respectively. Since knowledge distillation aims to use a larger model to improve a smaller one, we assume $|C_T| > |C_S|$.

Previous studies have shown that smaller neural networks can be initialized with subsets of weights extracted from larger pretrained models [17]. For Tsetlin Machines, the closest analogue to hidden-layer features is the set of propositional clauses. Because these clauses are independently learned patterns, they can be directly transferred between models. Thus, a student TM_S can be initialized by copying up to $|C_S|$ clauses from the teacher TM_T, effectively providing pretrained structure. This constitutes a form of feature-based knowledge distillation.

To accomplish this, we introduce *IntelligentTransfer* (Algorithm 1), which selects the top-$|C_S|$ teacher clauses based on a combination of influence and diversity. Let ζ denote the number of output classes, and let $z \in [0, 1]$ represent the fraction of clauses allocated by weight, with $(1 - z)$ representing the fraction allocated by diversity.

Algorithm 1 initializes the student Tsetlin Machine using clauses learned by the teacher model. Since clauses encode propositional patterns discovered during training, transferring a subset of these clauses allows the student to inherit useful structural bias before learning begins. The proposed selection strategy balances two complementary criteria: clause importance, measured by clause weight, and clause diversity, measured by the proportion of active Tsetlin Automata within each clause. This prevents the student from inheriting only highly weighted but redundant clauses, while still prioritizing influential patterns. The result is a compact initialization that preserves both discriminative strength and feature coverage.

For weight-based selection, the teacher clauses in each class are sorted in descending order of their clause weights. Let λ_{weight} denote the index set of the top $\lfloor z \cdot |C_S| \rfloor$ clauses. The remaining clause indices form the set Ω_{remain}.

To compute diversity, we evaluate each remaining clause $(j, w_j) \in \Omega_{\text{remain}}$ for every class k. Let A_j^k denote the set of Tsetlin Automata (TAs) associated with clause j in class k. Define the TA activation ratio for clause j in class k as

$$a_j = \frac{|A_j^k|_1}{|A_j^k|_0 + |A_j^k|_1},$$

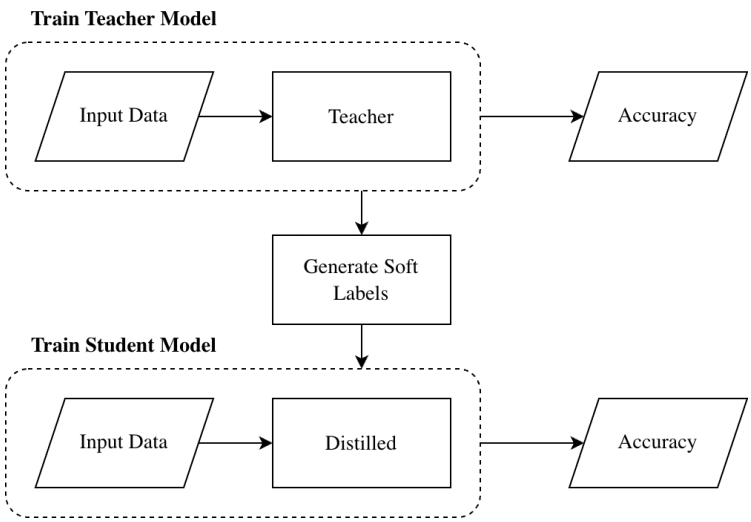


Fig. 2: Visualization of the training process using soft labels.

which reflects how often the clause includes its literals. Higher a_j values indicate clauses that engage with a broader portion of the input space. Next, we normalize the clause weight,

$$w'_j = \frac{w_j}{\max(W^k)}$$

where W^k is the set of clause weights for class k. Combining diversity and normalized weight yields

$$v_j = w'_j \cdot a_j$$

Each score is stored in a set D, which is then sorted in descending order. The top $(|C_S| - n_{\text{direct}})$ indices from D form the diversity-based clause set λ_{diverse}. The complete selection set is $\lambda_{\text{selected}} = \lambda_{\text{weight}} \cup \lambda_{\text{diverse}}$, which is transferred to the student.

Varying the weight-transfer parameter z changes the balance between weight-focused and diversity-focused selection. When $z = 0$, all transferred clauses are selected through diversity scoring. When $z = 1$, selection is based solely on clause weight. In our experiments, $z = 0.2$ provided an effective balance.

4.2 Soft Label Generation

In order to teach the student TM to mimic the teacher, the student must be trained with a probability distribution over the teacher's class predictions for each training sample. In traditional networks, these probability distributions (soft labels) are easily obtained from the output layer after applying the softmax function (see Figure 3). The soft labels indicate for a data point $\mathbf{x}_i$ how likely it belongs to each class, with the class having the highest probability being the predicted class. On the left side of Figure 3 this is visualized; the selected class is shown in red. In TMs, however, we use discrete clause voting rather than continuous activations, so a different approach is required.

Algorithm 1 IntelligentTransfer: Clause Initialization for Student Tsetlin Machine

Require: TM_T: trained teacher Tsetlin Machine
Require: TM_S: student Tsetlin Machine
Require: $z \in [0, 1]$: fraction of clauses selected by weight

function IntelligentTransfer(TM_T, TM_S, z)
  for each class $k \in \{1, \ldots, \zeta\}$ **do**
    $W^k \leftarrow$ clause weights of teacher for class k
    $A^k \leftarrow$ TA states associated with each clause in class k
    $C_S^k \leftarrow$ number of student clauses allocated to class k
    $\Omega \leftarrow \{(j, W_j^k) : j = 1, \ldots, |W^k|\}$
    $n_{\text{direct}} \leftarrow \max(1, \lfloor z \cdot C_S^k \rfloor)$
    Sort Ω in descending order by weight
    $\lambda_{\text{weight}} \leftarrow$ first n_{direct} indices from Ω
    $\Omega_{\text{remain}} \leftarrow \Omega \setminus \lambda_{\text{weight}}$
    $D \leftarrow \emptyset$
    for each (j, w_j) in Ω_{remain} **do**
      $a_j \leftarrow \frac{|A_j^k|_1}{|A_j^k|_0 + |A_j^k|_1}$
      $w'_j \leftarrow \frac{w_j}{\max(W^k)}$
      $v_j \leftarrow w'_j \cdot a_j$
      Append (j, v_j) to D
    end for
    Sort D in descending order by score
    $n_{\text{diverse}} \leftarrow C_S^k - n_{\text{direct}}$
    $\lambda_{\text{diverse}} \leftarrow$ first n_{diverse} indices from D
    $\lambda_{\text{selected}} \leftarrow \lambda_{\text{weight}} \cup \lambda_{\text{diverse}}$
    for $m \leftarrow 1$ to $|\lambda_{\text{selected}}|$ **do**
      Transfer clause and weight from teacher at index $\lambda_{\text{selected}}[m]$ to student position m
    end for
  end for
end function

Recall that in a traditional multi-class Tsetlin Machine, classification is determined by computing class sums (the sum of positive and negative weighted clause votes). Let $|C|$ be the number of clauses and ζ the number of classes in a Tsetlin Machine. For a weighted multi-class Tsetlin Machine, the class sums $C \in \mathbb{R}^\zeta$ over the ζ classes are computed as shown in Equation (1):

$$\mathbb{C}_k = \sum_{j=1}^{|C|/2} w_{j,k}^+ C_{j,k}^+ - \sum_{j=1}^{|C|/2} w_{j,k}^- C_{j,k}^-, \quad \forall k \in \{1, \ldots, \zeta\}. \tag{1}$$

In order to convert these class sums into valid soft labels, we apply the procedure given in Algorithm 2. These class sums contain valuable information about the teacher's confidence across all classes, not just the top class. However, they cannot

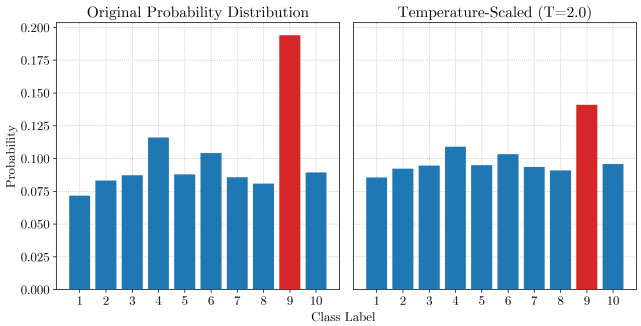


Fig. 3: Visualization of soft label probability distributions generated by Algorithm 2. Left: Temperature $\tau = 1.0$ (as generated). Center: $\tau = 2.0$ (softer, more uniform). Right: $\tau = 0.5$ (sharper, peaked). The red bar indicates the predicted class. Higher temperatures expose more information about class relationships by flattening the distribution. Temperature scaling is applied during training (Algorithm 3), not during soft label generation.

be directly used as probability distributions because they can be negative (due to negative clause votes), they are unbounded, and may not sum to one.

Algorithm 2 converts raw, unclamped class sums produced by a teacher Tsetlin Machine into probability distributions suitable for knowledge distillation. Unlike neural networks, Tsetlin Machines produce discrete and potentially unbounded class sums that may be negative and are not directly comparable across samples. The algorithm therefore applies a sequence of transformations to preserve relative class confidence while ensuring numerical stability: class sums are first shifted to remove negative values, then normalized on a per-sample basis, and finally transformed into a probability distribution using a softmax function. In degenerate cases where all class sums are equal, the procedure yields a uniform distribution, reflecting maximal uncertainty in the teacher's prediction.

To address these issues, Algorithm 2 applies a sequence of transformations designed to preserve relative class confidence while ensuring numerical stability. First, a per sample shifting operation removes negative values without altering the ordering of class scores. Next, per sample normalization scales class sums to a comparable range, preventing dominance by samples with large magnitude scores. The exponential transformation emphasizes relative differences between classes, and the final normalization step produces valid probability distributions that sum to one. Together, these steps enable stable and meaningful soft labels suitable for distillation in the absence of continuous logits or backpropagation.

Here S_{ij} can be interpreted as the teacher's estimated probability $\mathbb{P}(y_i = j \mid \mathbf{x}_i)$. Each row S_i is a valid probability distribution over classes.

For example, suppose for a 3-class TM we obtain raw class sums

$$\mathbb{C} = [-50,\ 120,\ -30].$$

Algorithm 2 Generate Soft Labels from Tsetlin Machine Class Sums

Require: $\mathbb{C} \in \mathbb{R}^{N \times \zeta}$: raw (unclamped) class sums for N examples across ζ classes

function GetSoftLabels($\mathbb{C}$) ▷ Phase 1: Make class sums non-negative
  $c^i_{\min} = \min_{j \in \{1,\ldots,\zeta\}} \mathbb{C}_{ij}, \quad \forall i = 1, \ldots, N$
  $\hat{\mathbb{C}}_{ij} = \mathbb{C}_{ij} + |c^i_{\min}|, \quad \forall i, j$ ▷ Now $\hat{\mathbb{C}}_{ij} \geq 0$
  ▷ Phase 2: Normalize per example to [0,1]
  $m_i = \max_{j \in \{1,\ldots,\zeta\}} \hat{\mathbb{C}}_{ij}, \quad \forall i$
  if $m_i > 0$ **then**
    $\bar{\mathbb{C}}_{ij} = \hat{\mathbb{C}}_{ij}/m_i, \quad \forall j$ ▷ So that $\bar{\mathbb{C}}_{ij} \in [0, 1]$ and the maximum equals 1
  else
    $\bar{\mathbb{C}}_{ij} = 0, \quad \forall j$ ▷ In the degenerate case where all raw sums are equal
  end if
  ▷ Phase 3: Exponentiate and normalize to produce a probability distribution
  $P_{ij} = \exp\left(\bar{\mathbb{C}}_{ij}\right), \quad \forall i, j$
  $S_{ij} = \dfrac{P_{ij}}{\sum_{k=1}^{\zeta} P_{ik}}, \quad \forall i, j$
  return $S \in \mathbb{R}^{N \times \zeta}$
end function

Then Phase 1 (shift) yields

$$\hat{\mathbb{C}} = [0,\ 170,\ 20].$$

Phase 2 (normalize) yields

$$\bar{\mathbb{C}} = [0,\ 1.0,\ 0.118],$$

Phase 3 (softmax) yields

$$P = [\exp(0),\ \exp(1.0),\ \exp(0.118)] = [1,\ 2.718,\ 1.125],$$

and finally the normalized soft labels are

$$S \approx [0.204,\ 0.554,\ 0.229].$$

Thus the teacher assigns approximately 55.4% probability to class 2, 22.9% to class 3, and 20.4% to class 1 — not just the single top class but a full distribution reflecting confidence and relative class plausibility.

The "dark knowledge" contained in a teacher model is transferable to the student through its inter-class similarity structure [1]. In a TM, that inter-class similarity is encoded in the relative ordering and magnitude differences of TM class sums. Because shifting by a constant (Phase 1), normalizing by a positive scalar (Phase 2), and applying softmax (Phase 3) each retain the relative ordering of their inputs, their composition likewise preserves rank ordering. Therefore, the soft labels produced by Algorithm 2 suitably preserve the dark knowledge contained in the teacher.

Our approach differs from standard logit-based softmax (often used in neural networks) by introducing shifting and scaling before exponentiation. These steps ensure non-negativity, avoid numerical instability due to large or negative sums, and preserve

relative differences in class votes. We apply any temperature scaling only during the fit stage (Algorithm 3), not during soft-label generation, which simplifies reuse of soft labels across different training regimes.

This procedure enables the student model to learn not only the teacher's top prediction but its confidence structure across classes, potentially transferring richer "dark knowledge" as shown to improve generalization in various distillation settings (e.g., knowledge distillation literature).

4.3 Improved Training

To use the soft labels generated by the teacher in Algorithm 2, we modify the standard Tsetlin Machine update step. Algorithm 3 describes the training procedure for the student Tsetlin Machine using both hard labels and soft labels provided by the teacher. During training, the student receives standard clause feedback derived from ground-truth labels, while additional guidance is incorporated through probability distributions generated from the teacher's class sums. A balance parameter (α) controls the relative influence of hard and soft supervision, allowing the student to learn both the target labels and the teacher's confidence structure. This approach preserves the original Tsetlin Machine learning dynamics while enriching the training signal with distribution-level information.

Training consists of two phases: reinforcing the positive class and reinforcing the negative classes. Recall that a multi-class Tsetlin Machine can be viewed as a team of ζ TMs, one for each class. In standard multi-class TM training, Type I feedback is sent to the true class c, and Type II feedback is sent to a random negative class $\neq c$.

We propose a novel fit method for use in Tsetlin Machines, described in Algorithm 3. As in traditional TM training, the algorithm takes training data $X \in \mathbb{R}^{N \times n}$, true labels $y \in \mathbb{R}^N$, the number of training examples N, and the number of epochs E. We introduce three additional parameters: soft labels $S \in \mathbb{R}^{N \times \zeta}$, balance $\alpha \in [0, 1]$, and distribution temperature $\tau \in (0, \infty)$. The balance parameter α controls the influence of soft labels in training, while the temperature τ adjusts the entropy of the soft labels. Figure 2 illustrates the data pathway for training the student model.

First, for each epoch e and example i, the soft label distributions are adjusted by temperature τ^2 and re-normalized. This modifies the entropy of the soft labels: a higher $\tau > 1$ increases entropy, moving the distribution toward uniform, while $\tau < 1$ decreases entropy, emphasizing the true class. This effect is shown in Figure 3.

In Algorithm 3, Type I feedback is applied to the true class $y_i = c$ with probability α, allowing the student to learn from ambiguous examples and mimic the teacher. For each class k, the base feedback probability $\phi = (1 - \alpha)p_{i,k}$ is computed. Classes with $\phi < 0.001$ are skipped for efficiency. The feedback type is determined by the soft label: $p_{i,k} > 0.5$ indicates a possible positive class (Type I), otherwise Type II. The feedback probability is then adjusted by confidence and temperature before stochastic application. This process is repeated for each class, example, and epoch, enabling the student model to integrate both teacher soft labels and ground truth information.

Algorithm 3 Tsetlin Machine fit method using soft label distributions

Require: $X \in \mathbb{R}^{N \times n}$: Training examples
Require: $y \in \mathbb{R}^{N}$: True class labels
Require: $S \in \mathbb{R}^{N \times \zeta}$: Teacher probability distributions (soft labels)
Require: N: Number of training examples
Require: E: Number of epochs
Require: $\alpha \in [0, 1]$: Balance between hard/soft labels (0=pure soft, 1=pure hard)
Require: $\tau \in (0, \infty)$: Temperature for sharpening/softening distributions

function FitEnhanced$(X, y, S, N, E, \alpha, \tau)$
  for each epoch $e \in \{1, \ldots, E\}$ **do**
    for each example $i \in \{1, \ldots, N\}$ **do**
      $c \leftarrow y_i$ ▷ True class label for current example
      $p_i \leftarrow S_i^{1/\tau^2}$ ▷ Apply temperature scaling
      $p_i \leftarrow p_i / \sum p_i$ ▷ Renormalize to probability distribution
      ▷ Phase 1: Hard Label Training
      $u \sim \mathcal{U}(0, 1)$
      if $u \leq \alpha$ **then**
        TM_c.update$(X_i, \texttt{Type I})$
      end if
      ▷ Phase 2: Soft Label Training
      for each class $k \in \{1, \ldots, \zeta\}$ **do** ▷ Skip true class if already trained with hard labels
        if $k = c$ **and** $\alpha > 0$ **then**
          continue
        end if
        $\phi \leftarrow (1 - \alpha)\, p_{i,k}$ ▷ Base feedback probability
        if $\phi < 0.001$ **then** ▷ Skip unlikely classes
          continue
        end if
        if $p_{i,k} > 0.5$ **then**
          $f \leftarrow \texttt{Type I}$ ▷ Positive feedback
          $\phi \leftarrow \phi(1 + p_{i,k}\tau)$
        else
          $f \leftarrow \texttt{Type II}$ ▷ Negative feedback
          $\phi \leftarrow \phi(1 + (1 - p_{i,k})\tau)$
        end if
        $u \sim \mathcal{U}(0, 1)$ ▷ Independent random draw
        if $u \leq \phi$ **then**
          TM_k.update(X_i, f)
        end if
      end for
    end for
  end for
end function

5 Experiment and Results

In this section, we explore how using a feature- and response-based teacher-student knowledge distillation model affects accuracy and performance across several datasets. We test both approaches described in the previous section to determine which performs best. For both approaches, we compare training and testing accuracy and runtime to evaluate effectiveness. All experiments are conducted using a parallel Tsetlin Machine implementation [1].

5.1 Datasets

We use four standard benchmark datasets: MNIST, KMNIST, EMNIST, and IMDB.

5.2 Dataset Descriptions

5.2.1 MNIST

MNIST (Modified National Institute of Standards and Technology) [40] is a widely used dataset containing 70,000 samples of handwritten digits. It is split into 60,000 training images and 10,000 testing images. Each image is a grayscale digit in $\{0, 1, ..., 9\}$ and has size 28×28 pixels.

5.2.2 Kuzushiji-MNIST (KMNIST)

Kuzushiji-MNIST [41] contains 70,000 grayscale images of historical Japanese characters, with 60,000 training images and 10,000 test images. Each image has size 28×28 pixels and there are 10 classes. KMNIST provides an alternative to MNIST for evaluating classification performance on non-Latin characters.

5.2.3 Extended-MNIST (EMNIST)

EMNIST [42] contains 145,600 grayscale 28×28 images. It consists of 124,800 training images and 20,800 testing images across 26 classes, one for each letter of the alphabet.

5.2.4 IMDB

The IMDB dataset [43] is a text classification dataset for sentiment analysis over movie reviews. It contains 25,000 highly polarized movie reviews for training and 25,000 for testing. There are two classes: positive and negative.

5.3 Dataset Preprocessing

For the image datasets, all grayscale images are binarized at a threshold of 75/255 (0.3 normalized). This threshold provides a balance between preserving edge information and reducing noise across all three image datasets. Pixels with intensity ≤ 75 are set to 0, and pixels with intensity > 75 are set to 1.

IMDB textual data is preprocessed into n-grams and converted into bit representations, resulting in a total of 5,000 features per sample.

[1] https://github.com/ckinateder/pyTsetlinMachineParallel

Table 1: Dataset Information

Dataset	$\|X_{train}\|$	$\|X_{test}\|$	$\|L\|$	ζ	Type
EMNIST	124800	20800	784	26	Image
MNIST	60000	10000	784	10	Image
KMNIST	60000	10000	784	10	Image
IMDB	25000	25000	5000	2	Text

5.4 Experiment Design

Our experiments follow a knowledge distillation scenario similar to neural networks. We have a baseline model TM_B, a teacher model TM_T, and a student model TM_S. All three models are weighted Tsetlin Machines, and share the same parameters aside from clause count. The baseline and student models are assigned fewer clauses than the teacher (in our experiments, between 2 and 10 times fewer), and the student has the same number of clauses as the baseline. The baseline serves as a reference for what the student would achieve without the effect our proposed algorithms (DKD). Because the baseline and student models have the same size, they should have roughly the same computational cost.

Let E_S and E_T denote the number of student and teacher epochs, respectively. Let $E_{T+S} = E_T + E_S$ denote the total number of epochs. The choice of epoch counts (Table 2) was determined through preliminary convergence analysis on validation data.

First, the baseline model TM_B is trained on the dataset for E_{T+S} epochs, providing a reference accuracy for a Tsetlin Machine with C_S clauses and E_{T+S} epochs. Next, the teacher model TM_T is trained with C_T clauses for E_{T+S} epochs.

Once the baselines are established, we reinitialize the teacher model TM_T from the checkpoint at E_T epochs to avoid giving the student model an unfair advantage. We then initialize the student model TM_S with the clauses of the trained teacher using Algorithm 1. Soft labels are generated from the teacher over the training dataset using Algorithm 2. Finally, the student model is trained for E_S epochs on the training dataset and the soft labels from the teacher using Algorithm 3. Because Tsetlin Machines are stochastic, each experiment is repeated K times and results are averaged, reporting mean and standard deviation across runs.

We evaluate the statistical siginificance of observed improvements by conducting paired t-tests across K independent trials, comparing the distilled student against the baseline. We use a one tailed paired t-test to measure accuracy (H_1: student $>$ baseline) and a two tailed test on training and inference time (H_1:), reflecting our directional hypothesis on accuracy and neutral effect on computational time. Significance thresholds are set at the conventional $p < 0.05$, $p < 0.01$, and $p < 0.001$ and reported as *, **, and ***, respectively.

The experimental design focuses on isolating the effect of distribution-enhanced supervision and clause-level transfer under fixed architectural settings. Accordingly, comparisons are limited to teacher, baseline, and student models that share identical hyperparameters aside from clause count. Alternative approaches such as extended training of smaller Tsetlin Machines, regularized TMs, and explicit clause pruning

Table 2: Experiment Hyperparameters

Dataset	$\|C_T\|$	$\|C_S\|$	T	s	τ	α	z	E_T	E_S	K
MNIST	1000	100	10	4.0	3.0	0.5	0.3	120	240	30
KMNIST	2000	200	100	8.2	4.0	0.5	0.3	120	240	30
EMNIST	1000	100	100	4.0	4.0	0.5	0.2	120	240	30
IMDB	8000	4000	6000	7.0	3.0	0.5	0.2	30	60	10

without distillation are not evaluated experimentally in this study. These comparisons represent important directions for future work.

For data collection, we measure average accuracies over E_{T+S} epochs. Training and testing times are recorded for the baseline and teacher over E_{T+S} epochs, and for the student model over E_S epochs. This reflects that the student model begins training after E_T epochs with a smaller architecture than the teacher. The time for generating soft labels and clause outputs is not included, as it is a one-time cost approximately equal to the time of one training epoch. All times are reported in seconds, and all accuracies are reported in percentages.

Hyperparameters for our distribution-enhanced experiments are shown in Table 2. Temperature selection (τ in Table 2) was determined through preliminary grid search experiments over the range $[1.0, 2.0, 3.0, 4.0, 5.0]$. We found that $\tau \in [3.0, 4.0]$ consistently produced the best results across datasets, outperforming the vanilla softmax ($\tau = 1.0$). This suggests that softer distributions, which reveal more information about class relationships, are particularly valuable for Tsetlin Machines, likely because they help guide discrete propositional logic clauses toward capturing inter-class boundaries more effectively. We denote the internal TM threshold and specificity as T and s, respectively [4]. Each dataset has its own T and s values, and all models within that respective dataset share the same T and s.

Overall, we observe across all datasets that the proposed knowledge distillation approach consistently improves the accuracy of the student model relative to a non-distilled baseline under identical architectural constraints, while maintaining comparable training and inference costs.

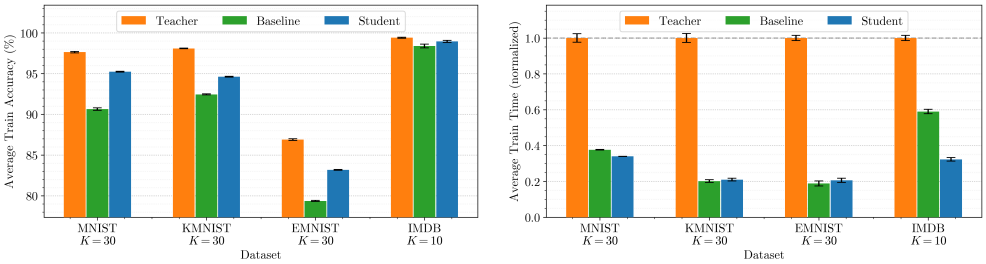
5.5 Training Accuracy and Performance Analysis

The experimental results indicate that teacher-student knowledge distillation consistently improves the training accuracy of a smaller model with minimal overhead. Clause transfer from the teacher to the student model, combined with distribution-enhanced fitting, allows the student model to achieve higher accuracy without increasing training time. These results are summarized in Table 3 and illustrated in Figures 4a and 4b. Note that the average training times in Figure 4b are normalized to allow fair comparison across datasets of different sizes.

Training efficiency can also be visualized in an efficiency plot (Figure 5b). Points to the left of the diagonal line represent a “free lunch” region, where accuracy increases without a proportional increase in execution time. This reflects the core idea of knowledge distillation: improving accuracy without increasing training or inference cost by the normalized equivalent amount.

Table 3: Training accuracy and epoch time for the teacher (Acc_T, $\mathcal{T}_T$), baseline (Acc_B, $\mathcal{T}_B$), and student (Acc_S, $\mathcal{T}_S$) models. δ represents the student minus baseline difference. Statistical significance is displayed for the paired t-test results via stars: $^*p < 0.05$, $^{**}p < 0.01$, $^{***}p < 0.001$ (one-tailed for accuracy, two-tailed for time). All values are reported as mean $\pm$ standard deviation across K trials. Each TM is weighted, and the student and the baseline are parametrically identical. Only the student model uses our DKD algorithm.

Metric	MNIST	KMNIST	EMNIST	IMDB
Acc_T	97.63 ± 0.08	98.09 ± 0.04	86.89 ± 0.09	99.41 ± 0.07
Acc_B	90.63 ± 0.16	92.45 ± 0.06	79.36 ± 0.06	98.39 ± 0.24
Acc_S (DKD)	95.24 ± 0.06	94.63 ± 0.05	83.17 ± 0.06	98.96 ± 0.13
Δ ($Acc_S - Acc_B$)	+4.61***	+2.18***	+3.81***	+0.57***
$\mathcal{T}_T$	0.70 ± 0.02	1.66 ± 0.04	3.09 ± 0.05	28.95 ± 0.42
$\mathcal{T}_B$	0.26 ± 0.00	0.33 ± 0.01	0.58 ± 0.04	17.07 ± 0.35
$\mathcal{T}_S$ (DKD)	0.24 ± 0.00	0.35 ± 0.01	0.64 ± 0.04	9.35 ± 0.29
Δ ($\mathcal{T}_S - \mathcal{T}_B$)	-0.03***	+0.01***	+0.05***	-7.72***

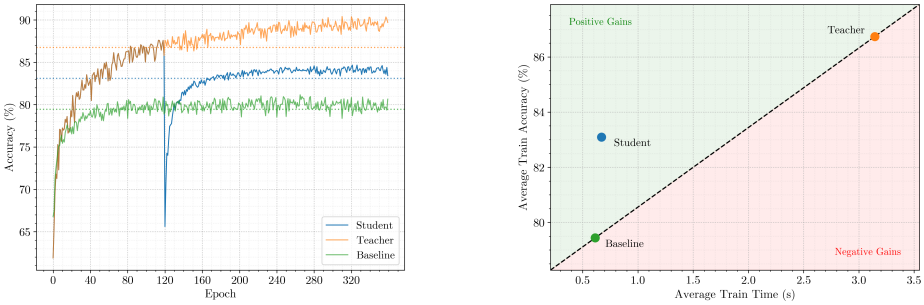


(a) Average training accuracy on each dataset using DKD.

(b) Average epoch training time $\mathcal{T}'$ on each dataset using DKD.

Fig. 4: Training accuracy and epoch time on each dataset using DKD.

For MNIST, the student model TM_S increases training accuracy over the baseline by approximately 5 percentage points, with reduced variance and slightly lower training time. On KMNIST, TM_S achieves modest gains, increasing average accuracy by around 2 percentage points with similar epoch times. For the EMNIST dataset, training accuracy improves by nearly 4 percentage points compared to the baseline model, while maintaining similar epoch times. This is further illustrated in Figures 5a and 4b. On the IMDB dataset, TM_S achieves accuracy between the teacher and baseline while reducing the training time by half when compared to the baseline, and by almost four times when compared to the teacher. This is enabled by rapid literal pruning guided by the teacher, and IMDB results stabilize more quickly than the MNIST datasets, allowing for fewer epochs. Note that the neglible increase in training time for KMNIST and EMNIST occurs for the same reason.



(a) Training accuracy on EMNIST using DKD.

(b) Training efficiency on EMNIST using DKD.

Fig. 5: Training accuracy and efficiency on EMNIST using DKD.

For all datasets, the effects of DKD on training accuracy and epoch time are statistically significant at $p < 0.001$, as evidenced by our paired t-test on the baseline and student model.

The temporary drop in accuracy at $E = E_T$ in each plot reflects the student model's adjustment to clause transfer. The baseline model TM_B is trained from scratch, whereas the student model TM_S starts with $|C_S|$ clauses copied from TM_T after E_T epochs. The student model needs several epochs to re-balance clause contributions in the absence of the remaining teacher clauses. For example, in Figure 5a, it takes approximately 7 epochs for TM_S to reach the equilibrium accuracy of the baseline, which is fewer epochs than required for the baseline to reach the same level. This demonstrates that clause initialization combined with distribution-enhanced fitting enables efficient training of a smaller model.

5.6 Testing Accuracy and Performance Analysis

The results on the test datasets further confirm that our teacher-student knowledge distillation framework improves both accuracy and performance. Table 4 summarizes the testing performance of each model across all datasets.

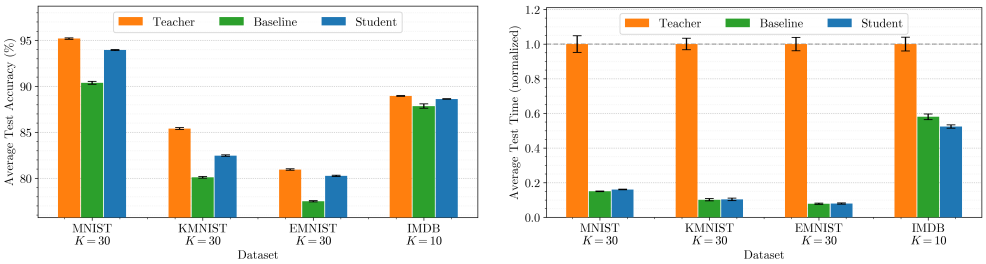
Figure 6a shows the average accuracy of each model for each dataset. As described in Table 4, each student model consistently outperforms its respective baseline model across the evaluated datasets.

Figure 6b shows the normalized average testing time per epoch relative to the teacher. Normalization allows a fair comparison across datasets of different sizes (see Table 1). We observe that each student model achieves testing performance similar to its respective baseline model while improving accuracy. As with the training analyis, we observe that all perfomance gains are statistically significant at $p < 0.001$.

For EMNIST, the student model demonstrates notable efficiency under the evaluated settings. As shown in Figure 7b, TM_S achieves near-teacher accuracy while reducing inference time by roughly 12×. The accuracy is comparable to multi-layer

Table 4: Testing accuracy and epoch time for the teacher (Acc_T, $\mathcal{T}_T$), baseline (Acc_B, $\mathcal{T}_B$), and student (Acc_S, $\mathcal{T}_S$) models. Δ represents the student minus baseline difference. Statistical significance is displayed for the paired t-test results via stars: $^*p < 0.05$, $^{**}p < 0.01$, $^{***}p < 0.001$ (one-tailed for accuracy, two-tailed for time). All values are reported as mean $\pm$ standard deviation across K trials. Each TM is weighted, and the student and the baseline are parametrically identical. Only the student model uses our DKD algorithm.

Metric	MNIST	KMNIST	EMNIST	IMDB
Acc_T	95.19 ± 0.08	85.41 ± 0.10	80.95 ± 0.09	88.96 ± 0.04
Acc_B	90.37 ± 0.16	80.11 ± 0.09	77.51 ± 0.07	87.85 ± 0.24
Acc_S (DKD)	93.95 ± 0.05	82.47 ± 0.09	80.26 ± 0.07	88.62 ± 0.03
Δ ($Acc_S - Acc_B$)	+3.58***	+2.36***	+2.75***	+0.78***
$\mathcal{T}_T$	0.38 ± 0.02	0.95 ± 0.03	3.13 ± 0.12	17.88 ± 0.71
$\mathcal{T}_B$	0.06 ± 0.00	0.10 ± 0.01	0.24 ± 0.01	10.37 ± 0.30
$\mathcal{T}_S$ (DKD)	0.06 ± 0.00	0.10 ± 0.01	0.25 ± 0.01	9.36 ± 0.17
Δ ($\mathcal{T}_S - \mathcal{T}_B$)	+0.00***	+0.00***	+0.00***	-1.00***



(a) Average testing accuracy on each dataset using DKD.

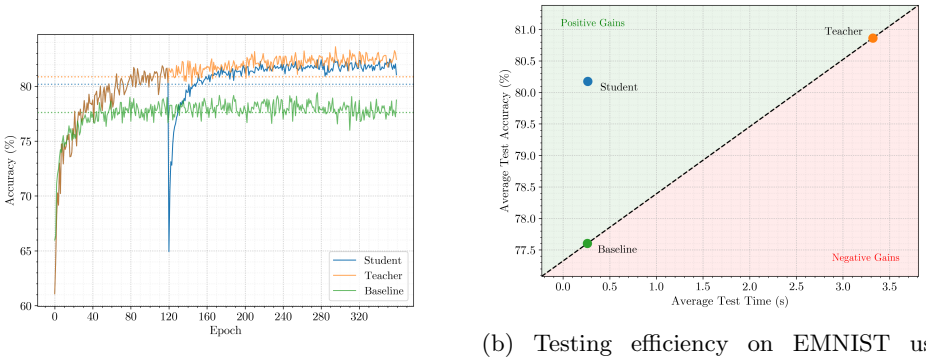
(b) Average testing epoch time $\mathcal{T}$ on each dataset using DKD.

Fig. 6: Testing accuracy and epoch time on each dataset using DKD.

Tsetlin Machines reported in [36], but with lower inference time and a reduced number of clauses (Figures 7a and 6b).

Tsetlin Machine memory consumption scales as $\mathcal{O}(\zeta|C|n)$. Because TM_T and TM_S are parametrically identical aside from clause count, the memory footprint of the student reduces by the teacher-to-student clause ratio: 10× for MNIST, KMNIST, and EMNIST, and by 2× for IMDB. This relationship is preserved in any TM implementation.

MNIST testing results reflect similar trends: TM_S improves accuracy over the baseline by approximately 3.5–5 percentage points while maintaining or slightly reducing testing time. On KMNIST, the student model exceeds the baseline by about 2.5 percentage points without increasing execution time. For IMDB, TM_S improves accuracy by roughly 1 percentage point while decreasing inference time compared to both baseline and teacher models.



(a) Testing accuracy on EMNIST using DKD.

(b) Testing efficiency on EMNIST using DKD.

Fig. 7: Testing accuracy and efficiency on EMNIST for teacher, baseline, and student models

The temporary drop in accuracy at E_T epochs is also observed in testing, for the same reasons described in Section 5.5.

5.7 Activation Analysis

The influence of the teacher on the student can be visualized through activation maps generated over the input data. Figure 8 shows an example on EMNIST test samples. For each pixel, green indicates an included positive feature (e.g., a white pixel), and red indicates an included negated feature (e.g., a black pixel). Pixel intensity reflects feature importance.

Teacher activation maps are denser than those of the baseline, as expected given the baseline has 10× fewer clauses. The student model, with the same number of clauses as the baseline, exhibits a slightly wider range of included features, leveraging the additional context provided by clause transfer. This contributes to the improved accuracy of the student model [2].

Even with identical model sizes, TM_S shows lower activation entropy (3.1 vs 3.6), indicating more focused and strategic feature usage, which provides insight into its improved accuracy. Note that these visualizations are limited to image datasets; text feature patterns are less spatially interpretable.

6 Limitations

While our Distribution-Enhanced Knowledge Distillation demonstrates consistent empirical improvements, several limitations warrant acknowledgment.

Our experiments focus on relatively simple, balanced datasets with moderate feature dimensionality (784–5000 features). Performance on high-dimensional problems,

[2] Activation maps are straightforward to generate for image datasets but are less interpretable for textual datasets such as IMDB.

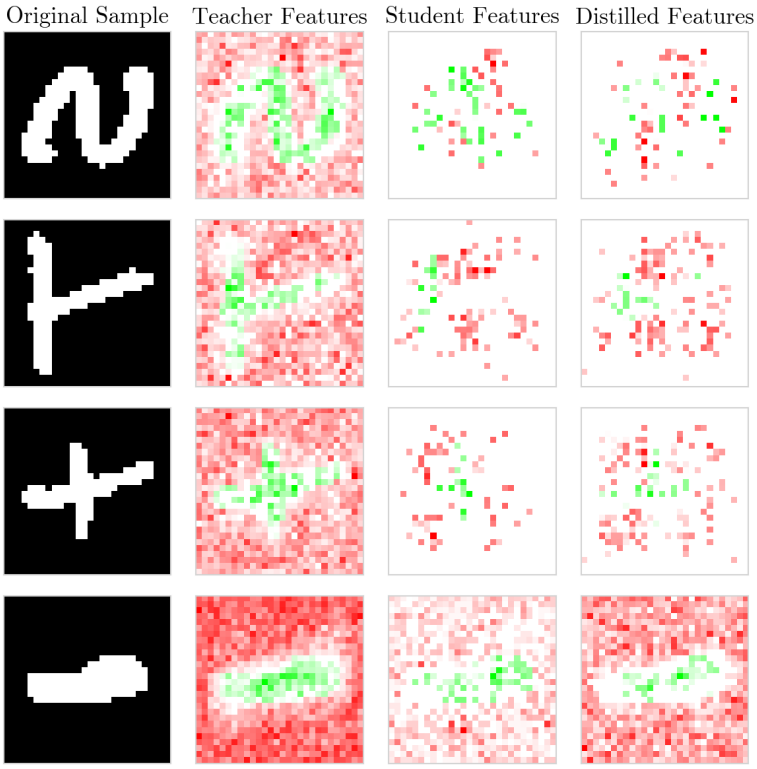


Fig. 8: Activation maps for TM_T, TM_B, and TM_S on four EMNIST test samples.

severely imbalanced classes, or complex domains (e.g., medical imaging, time-series) remains unexplored. Notably, IMDB shows a modest improvement of 0.57 percentage points, suggesting that distillation effectiveness may vary across domains.

We do not compare against simpler compression alternatives such as direct clause pruning from the teacher. Additionally, explicit memory consumption measurements and actual edge device deployment tests are not included in this study; however, such measurements would only be relevant to the specific tested Tsetlin Machine implementation, which is why it was omitted. The clause count of each model serves as a platform-agnostic representative basis for memory consumption.

Our approach lacks formal analysis of approximation bounds or information-theoretic characterization of what "dark knowledge" the soft labels capture in propositional logic systems. Understanding why temperature values $\tau \in [3.0, 4.0]$ work well for TMs, compared to $\tau \approx 1$–2 typical in neural networks, requires further investigation.

Further research could be directed at optimizing the balance α, weight transfer z, and temperature τ parameters, through automated hyperparameter search methods, or exploring adaptive temperature scheduling during training similar to curriculum

learning approaches. Investigating whether temperature should vary by dataset characteristics (e.g., number of classes, class imbalance) could yield additional insights. The clause transfer algorithm could also be improved by considering clause coverage on all features, not just diversity and weight. Instead of solely using offline distillation as shown in this paper, further research could also be directed at comparing the effects of online and offline distillation. We would also explore the application of our algorithm in other domains like few-shot learning [44–46] and bioinformatics [47], where data is few and knowledge transfer can help training these models with limited data.

7 Conclusion

Distribution-enhanced knowledge distillation substantially accelerates Tsetlin Machine training and inference while improving accuracy. In all tested datasets, TMs trained with DKD greatly surpass the performance of TMs of the same size trained naively. Activation map analysis confirms that the student model leverages teacher clause context to strategically select features. Training and inference times are reduced, demonstrating practical efficiency.

During the preparation of this work the author(s) used Claude and ChatGPT in order to fix grammatical and syntax errors and clarify the material in the paper. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the published article.

References

[1] Hinton, G., Vinyals, O., Dean, J.: Distilling the Knowledge in a Neural Network (2015). https://arxiv.org/abs/1503.02531

[2] DeepSeek-AI, et al.: DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning (2025). https://arxiv.org/abs/2501.12948

[3] Matsuyama, K., Anjum, U., Matsuyama, S., Shoda, T., Zhan, J.: Adaptive temperature based on logits correlation in knowledge distillation. arXiv preprint arXiv:2503.09030 (2025)

[4] Granmo, O.-C.: The tsetlin machine–a game theoretic bandit driven approach to optimal pattern recognition with propositional logic. arXiv preprint arXiv:1804.01508 (2018)

[5] Abeyrathna, K.D., Bhattarai, B., Goodwin, M., Gorji, S.R., Granmo, O.-C., Jiao, L., Saha, R., Yadav, R.K.: Massively parallel and asynchronous tsetlin machine architecture supporting almost constant-time scaling. In: International Conference on Machine Learning, pp. 10–20 (2021). PMLR

[6] Lei, J., Wheeldon, A., Shafik, R., Yakovlev, A., Granmo, O.-C.: From arithmetic to logic based ai: A comparative analysis of neural networks and tsetlin machine.

In: 2020 27th IEEE International Conference on Electronics, Circuits and Systems (ICECS), pp. 1–4 (2020). IEEE

[7] Xu, F., Uszkoreit, H., Du, Y., Fan, W., Zhao, D., Zhu, J.: Explainable ai: A brief survey on history, research areas, approaches and challenges. In: Natural Language Processing and Chinese Computing: 8th cCF International Conference, NLPCC 2019, Dunhuang, China, October 9–14, 2019, Proceedings, Part II 8, pp. 563–574 (2019). Springer

[8] Došilović, F.K., Brčić, M., Hlupić, N.: Explainable artificial intelligence: A survey. In: 2018 41st International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO), pp. 0210–0215 (2018). IEEE

[9] Anjum, U., Zadorozhny, V., Krishnamurthy, P.: Localization of unidentified events with raw microblogging data. Online Social Networks and Media **29**, 100209 (2022)

[10] Anjum, U., Zadorozhny, V., Krishnamurthy, P.: Localization of events using neural networks in twitter data. 20th Annual Global Conference on Information Systems for Crisis Response and Management (ISCRAM 2023), May 28 - May 31, 2023, Omaha, Nebraska, USA, 909–919 (2023)

[11] Zhang, J., Cheng, L., Li, C., Li, Y., He, G., Xu, N., Lian, Y.: A low-latency fpga implementation for real-time object detection. In: 2021 IEEE International Symposium on Circuits and Systems (ISCAS), pp. 1–5 (2021). IEEE

[12] Gao, M., Shen, Y., Li, Q., Yan, J., Wan, L., Lin, D., Loy, C.C., Tang, X.: An Embarrassingly Simple Approach for Knowledge Distillation (2019). https://arxiv.org/abs/1812.01819

[13] Tang, R., Lu, Y., Lin, J.: Natural language generation for effective knowledge distillation. In: Cherry, C., Durrett, G., Foster, G., Haffari, R., Khadivi, S., Peng, N., Ren, X., Swayamdipta, S. (eds.) Proceedings of the 2nd Workshop on Deep Learning Approaches for Low-Resource NLP (DeepLo 2019), pp. 202–208. Association for Computational Linguistics, Hong Kong, China (2019). https://doi.org/10.18653/v1/D19-6122 . https://aclanthology.org/D19-6122/

[14] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I.: Language models are unsupervised multitask learners. OpenAI Blog (2019)

[15] Cui, Z., Ke, R., Pu, Z., Wang, Y.: Deep Bidirectional and Unidirectional LSTM Recurrent Neural Network for Network-wide Traffic Speed Prediction (2019). https://arxiv.org/abs/1801.02143

[16] Zhang, L., Song, J., Gao, A., Chen, J., Bao, C., Ma, K.: Be Your Own Teacher: Improve the Performance of Convolutional Neural Networks via Self Distillation

(2019). https://arxiv.org/abs/1905.08094

[17] Xu, Z., Chen, Y., Vishniakov, K., Yin, Y., Shen, Z., Darrell, T., Liu, L., Liu, Z.: Initializing Models with Larger Ones (2023). https://arxiv.org/abs/2311.18823

[18] Bouneffouf, D.: Multi-armed Bandit Problem and Application. Djallel Bouneffouf, ??? (2023)

[19] Bouneffouf, D., Rish, I., Aggarwal, C.: Survey on applications of multi-armed and contextual bandits. In: 2020 IEEE Congress on Evolutionary Computation (CEC), pp. 1–8 (2020). IEEE

[20] Seraj, R., Sharma, J., Granmo, O.-C.: Tsetlin machine for solving contextual bandit problems. Advances in Neural Information Processing Systems **35**, 30194–30205 (2022)

[21] Granmo, O.-C., Glimsdal, S., Jiao, L., Goodwin, M., Omlin, C.W., Berge, G.T.: The convolutional tsetlin machine. arXiv preprint arXiv:1905.09688 (2019)

[22] Tunheim, S.A., Jiao, L., Shafik, R., Yakovlev, A., Granmo, O.-C.: Convolutional tsetlin machine-based training and inference accelerator for 2-d pattern classification. Microprocessors and Microsystems **103**, 104949 (2023)

[23] O'shea, K., Nash, R.: An introduction to convolutional neural networks. arXiv preprint arXiv:1511.08458 (2015)

[24] Darshana Abeyrathna, K., Granmo, O.-C., Zhang, X., Jiao, L., Goodwin, M.: The regression tsetlin machine: a novel approach to interpretable nonlinear regression. Philosophical Transactions of the Royal Society A **378**(2164), 20190165 (2020)

[25] Glimsdal, S., Granmo, O.-C.: Coalesced multi-output tsetlin machines with clause sharing. arXiv preprint arXiv:2108.07594 (2021)

[26] Oommen, B.J.: Stochastic searching on the line and its applications to parameter learning in nonlinear optimization. IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics) **27**(4), 733–739 (1997)

[27] Bhattarai, B., Granmo, O.-C., Jiao, L., Yadav, R., Sharma, J.: Tsetlin machine embedding: Representing words using logical expressions. arXiv preprint arXiv:2301.00709 (2023)

[28] Saha, R., Granmo, O.-C., Zadorozhny, V.I., Goodwin, M.: A relational tsetlin machine with applications to natural language understanding. Journal of Intelligent Information Systems, 1–28 (2022)

[29] Saha, R., Granmo, O.-C., Goodwin, M.: Using tsetlin machine to discover interpretable rules in natural language processing applications. Expert Systems **40**(4), 12873 (2023)

[30] Berge, G.T., Granmo, O.-C., Tveit, T.O., Goodwin, M., Jiao, L., Matheussen, B.V.: Using the tsetlin machine to learn human-interpretable rules for high-accuracy text categorization with medical applications. IEEE Access **7**, 115134–115146 (2019)

[31] Yadav, R.K., Jiao, L., Granmo, O.-C., Goodwin, M.: Human-Level Interpretable Learning for Aspect-Based Sentiment Analysis. In: The Thirty-Fifth AAAI Conference on Artificial Intelligence (AAAI-21) (2021). AAAI

[32] Abouzeid, A., Granmo, O.-C., Goodwin, M., Webersik, C.: Label-critic tsetlin machine: A novel self-supervised learning scheme for interpretable clustering. In: 2022 International Symposium on the Tsetlin Machine (ISTM), pp. 41–48 (2022). IEEE

[33] Phoulady, A., Granmo, O.-C., Gorji, S.R., Phoulady, H.A.: The weighted tsetlin machine: compressed representations with weighted clauses. arXiv preprint arXiv:1911.12607 (2019)

[34] Anjum, U., Zhan, J.: A novel tsetlin machine with enhanced generalization. Electronics **13**(19), 3825 (2024)

[35] Gao, Y., Liu, W., Lombardi, F.: Design and implementation of an approximate softmax layer for deep neural networks. In: 2020 IEEE International Symposium on Circuits and Systems (ISCAS), pp. 1–5 (2020). https://doi.org/10.1109/ISCAS45731.2020.9180870

[36] Gorbenko, A., Tarasyuk, O., Yakovlev, A., Shafik, R.: Multi-layer tsetlin machine: Architecture and performance evaluation. 2024 International Symposium on the Tsetlin Machine (ISTM), 29-30 Aug 2024, Pittsburgh, USA (2024)

[37] Bhattarai, B., Granmo, O.-C., Jiao, L., Andersen, P.-A., Tunheim, S.A., Shafik, R., Yakovlev, A.: Contracting Tsetlin Machine with Absorbing Automata (2023). https://arxiv.org/abs/2310.11481

[38] Abeyrathna, K.D., Abouzeid, A.A.O., Bhattarai, B., Giri, C., Glimsdal, S., Granmo, O.-C., Jiao, L., Saha, R., Sharma, J., Tunheim, S.A., et al.: Building concise logical patterns by constraining tsetlin machine clause size. arXiv preprint arXiv:2301.08190 (2023)

[39] Rahman, T., Maheshwari, S., Shafik, R., Yakovlev, A., Das, S.: Mileage: An automated optimal clause search paradigm for tsetlin machines. In: 2022 International Symposium on the Tsetlin Machine (ISTM), pp. 49–52 (2022). https://doi.org/10.1109/ISTM54910.2022.00017

[40] LeCun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. Proceedings of the IEEE **86**(11), 2278–2324 (1998)

[41] Clanuwat, T., Bober-Irizar, M., Kitamoto, A., Lamb, A., Yamamoto, K., Ha, D.: Deep learning for classical japanese literature. arXiv preprint arXiv:1812.01718 (2018)

[42] Cohen, G., Afshar, S., Tapson, J., Schaik, A.: EMNIST: an extension of MNIST to handwritten letters. CoRR **abs/1702.05373** (2017) 1702.05373

[43] Maas, A.L., Daly, R.E., Pham, P.T., Huang, D., Ng, A.Y., Potts, C.: Learning word vectors for sentiment analysis. In: Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies, pp. 142–150. Association for Computational Linguistics, Portland, Oregon, USA (2011). http://www.aclweb.org/anthology/P11-1015

[44] Anjum, U., Stockman, C., Luong, C., Zhan, F.: Using adaptive learning and momentum to improve generalization. Neural Computing and Applications, 1–28 (2025)

[45] Guo, X., Anjum, U., Zhan, J.: Cyberbully detection using BERT with augmented texts. In: Tsumoto, S., Ohsawa, Y., Chen, L., Poel, D.V., Hu, X., Motomura, Y., Takagi, T., Wu, L., Xie, Y., Abe, A., Raghavan, V. (eds.) IEEE International Conference on Big Data, Big Data 2022, Osaka, Japan, December 17-20, 2022, pp. 1246–1253. IEEE, ??? (2022). https://doi.org/10.1109/BIGDATA55660.2022.10020581 . https://doi.org/10.1109/BigData55660.2022.10020581

[46] Anjum, U., Zhan, J.: Weighted trilateration using domain knowledge for event localization in twitter data. In: Proceedings of the International ISCRAM Conference (2025)

[47] Woessner, A.E., Anjum, U., Salman, H., Lear, J., Turner, J.T., Campbell, R., Beaudry, L., Zhan, J., Cornett, L.E., Gauch, S., *et al.*: Identifying and training deep learning neural networks on biomedical-related datasets. Briefings in Bioinformatics **25**(Supplement_1), 232 (2024)